

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA0609	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Yuvasri B	Reg. No.:	192421231
Section / Batch:	BTECH IT	Date:	27/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NPComplete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Q40. Maximum Independent Edge Set (Matching Problem)

1. Problem Statement

Given a graph $G = (V, E)$, find the **largest possible set of edges** such that **no two selected edges share a common vertex**. This is called a **Maximum Matching**.

Input

- Set of vertices V
- Set of edges E

Output

- Maximum set of independent edges (matching)
- Maximum number of selected edges

2. Constraints

An edge can be selected only if:

- Its two vertices are not already used by another selected edge.
- No two selected edges have a common vertex.
- Each vertex can belong to **at most one selected edge**.

For example, if (A,B) is selected, edges (A,C) and (B,D) cannot be selected.

3. Effect of Constraints

The independence constraint reduces the number of feasible solutions because selecting one edge may prevent other edges from being selected.

For example:

A ----- B

| |

C ----- D

If (A,B) is selected, edges containing A or B cannot be selected. Therefore, the algorithm must consider alternative choices to obtain the largest matching.

4. Backtracking Algorithm

MAXIMUM-MATCHING(G)

Input:

$G = (V, E)$

V = set of vertices

E = set of edges

Output:

Maximum independent edge set

1. bestMatching $\leftarrow$ empty set

2. BACKTRACK(0, empty set, empty set)

3. Return bestMatching

BACKTRACK(i , currentMatching, usedVertices)

1. If $i = |E|$ then

 If $\text{size}(\text{currentMatching}) > \text{size}(\text{bestMatching})$ then

 bestMatching $\leftarrow$ currentMatching

 Return

2. $(u, v) \leftarrow E[i]$

3. // Option 1: Select the edge

4. If $u \notin \text{usedVertices}$ AND $v \notin \text{usedVertices}$ then

 Add (u, v) to `currentMatching`

 Add u and v to `usedVertices`

 BACKTRACK($i + 1$,
 `currentMatching`,
 `usedVertices`)

 Remove (u, v) from `currentMatching`

 Remove u and v from `usedVertices`

5. // Option 2: Do not select the edge

6. BACKTRACK($i + 1$,
 `currentMatching`,
 `usedVertices`)

5. Pruning

Pruning eliminates branches that cannot produce a better solution.

If $\text{size}(\text{currentMatching}) + (|E| - i)$

$\leq \text{size}(\text{bestMatching})$ then

 Return

This means that even if every remaining edge could be selected, the result would not improve the current best matching. Therefore, that branch is discarded.

The complete backtracking step becomes:

BACKTRACK(i , currentMatching, usedVertices)

1. If $\text{size}(\text{currentMatching}) + (|E| - i)$

$\leq \text{size}(\text{bestMatching})$ then

Return

2. If $i = |E|$ then

Update bestMatching if necessary

Return

3. $(u, v) \leftarrow E[i]$

4. If u and v are unused then

Select (u, v)

Mark u and v as used

BACKTRACK($i+1$, currentMatching, usedVertices)

Undo selection

5. BACKTRACK($i+1$, currentMatching, usedVertices)

6. Correctness

The algorithm is correct because for every edge it considers both possibilities:

1. **Include the edge**, if it does not conflict with an already selected edge.
2. **Exclude the edge**.

Thus, all valid combinations of independent edges are considered. The algorithm stores the largest valid combination found, so the final result is a maximum matching.

Pruning does not affect correctness because it removes only branches that cannot produce a solution better than the best solution already found.

7. Complexity

For a graph with $m = |E|$ edges, each edge has two choices: selected or not selected.

Therefore, the worst-case number of possibilities is:

$$O(2^m)$$

So the backtracking algorithm has **exponential worst-case time complexity**.

Space complexity is approximately:

$$O(m + |V|)$$

for the recursion stack, current matching, and used-vertex information.

Note: Maximum Matching in a general graph is actually solvable in **polynomial time** using specialized matching algorithms such as Edmonds' blossom algorithm. Therefore, it is **not NP-Hard in the ordinary graph-matching formulation**. The backtracking approach is exponential, but the matching problem itself is not NP-Hard. Problems such as **Maximum Independent Set** are NP-Hard.

8. Conclusion

Backtracking finds the maximum set of independent edges by exploring possible edge selections and rejecting conflicting edges immediately. Pruning further reduces unnecessary searches. Although this backtracking method has exponential worst-case complexity, the general maximum matching problem has more efficient polynomial-time algorithms.

Code of Conduct

I Yuvasri B /192421231 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yuvasri B

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				

Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____